PRIMOTONIC

Sample Vulnerability Reports

Redacted portfolio examples for client review

All client names, hostnames, tokens, account IDs, asset URLs, and third-party evidence links have been removed or replaced with placeholders. Payloads shown are inert demo markers only.

Prepared by	Primotonic Security / Full-Stack
Document type	Portfolio sample (redacted)
Classifications	Stored XSS (CWE-79)
Original reports	2 findings consolidated
Status	For demonstration — not live targets

Finding 1 — Stored XSS in CMS Promo Content

Severity: **Critical** | Type: Stored XSS (CWE-79) | Vector: Low-privilege operator → public player portal

Summary of Findings

A Stored Cross-Site Scripting (XSS) vulnerability was identified in a confidential content-management system (CMS) used to publish promotional content to a public player portal. An authenticated operator with limited privileges could submit HTML/JavaScript into promo content fields. The CMS accepted and stored the markup without adequate sanitization. When published, the content rendered on the public portal and executed in visiting players' browsers — enabling session/token theft and account takeover.

- CMS accepted HTML/Markdown via an authenticated GraphQL mutation without sanitizing dangerous markup.
- Stored content rendered on the public player portal and executed in the visitor context.
- A low-privilege **OPERATOR** role could update public-facing content fields (insufficient permission controls).
- Because session material was readable from JavaScript on the portal, stored XSS enabled credential theft and hijack.

Product Assets (redacted)

- Environment: Backoffice / CMS — `REDACTED_CMS_HOST`
- Public portal — `REDACTED_PLAYER_PORTAL`
- Admin API — GraphQL `POST /graphql` on `REDACTED_ADMIN_API`
- Affected: CMS content editor, `UpdateContentField` mutation, portal promo renderer

Vulnerability Type

- CWE-79: Stored Cross-Site Scripting (Persistent XSS)
- Root causes: unsanitized stored HTML; unsafe client rendering (e.g. innerHTML patterns); weak server-side validation; missing role granularity

Proof of Concept (sanitized)

High-level flow: update CMS promo content with an XSS marker; open the public promo page; observe execution in the player session context. Live hostnames, tokens, site IDs, asset URLs, and exfiltration endpoints are redacted below.

Steps

- Authenticate to Backoffice as OPERATOR and open CMS → Content Management.
- Intercept/save the `UpdateContentField` request and insert a demo payload into a content array value.
- Submit the update (payload persists in storage).
- Open the public portal promo page while authenticated as a player.
- Observe demo alert / marker execution (original research used a private webhook — omitted).

Example request (redacted)

```
POST /graphql HTTP/2
Host: REDACTED_ADMIN_API
Referer: https://REDACTED_CMS_HOST/
Role: OPERATOR
Authorization: Bearer REDACTED_TOKEN
Content-Type: application/json
Origin: https://REDACTED_CMS_HOST

{
```

```
  "operationName": "UpdateContentField",
  "query": "mutation UpdateContentField($input: UpdateContentFieldInput!) { updateContentField(
input: $input) { __typename } }",
  "variables": {
    "input": {
      "id": "promos",
      "site": "REDACTED_SITE_ID",
      "value": [{
        "0": "REDACTED_PROMO_TITLE",
        "1": "<p>General Mechanics</p><img src=x onerror=alert('DEMO_XSS')>",
        "2": "https://REDACTED_CDN/REDACTED_ASSET",
        "3": "https://REDACTED_CDN/REDACTED_ASSET",
        "6": false
      }]
    }
  }
}
```

Impact

- Persistent XSS on a public content channel affects every future viewer of the promo.
- Critical severity: low-privilege insider → player account takeover and direct financial risk.
- Possible session/token theft, unauthorized actions as the victim player, and brand/trust damage.

Remediation (recommended)

- Sanitize/allowlist HTML server-side before persist; encode on output.
- Avoid unsafe DOM sinks; use a vetted sanitizer if rich text is required.
- Enforce least-privilege: separate who can edit public content from read-only operators.
- Prefer HttpOnly cookies for session material; harden CSP.

Finding 2 — Stored XSS in Backoffice Prepared Messages

Severity: **High / Critical** | Type: Stored XSS (persistent) | Vector: Message author → privileged admin viewers

Summary of Findings

An attacker (or malicious insider) could inject HTML/JavaScript into a prepared-message content field in the backoffice messaging UI. When another administrator opened the message via “View”, the payload executed in their browser context — risking privileged account compromise, unauthorized actions, and data exfiltration. Message content was not sanitized or encoded before HTML render.

Product Assets (redacted)

- Environment: Backoffice (UAT / Production as applicable) — `REDACTED_ADMIN_HOST`
- Component: Internal / prepared message creation UI and message detail renderer
- Endpoint: GraphQL mutation `createInternalPreparedMessage` via `POST /graphql` on `REDACTED_TRIGGER_HOST`
- Note: Edge WAF did not reliably block long / encoded XSS payloads in testing

Vulnerability Type

- Stored Cross-Site Scripting (XSS) — persistent
- Inadequate output encoding / input validation on message title/content fields

Proof of Concept (sanitized)

- Open Admin Panel → Internal Messaging → create prepared message.
- Insert a demo payload into content (or title), e.g. `<img src=x onerror="alert('DEMO_XSS')">`
- Select any recipient(s) and submit.
- Message appears in Prepared Messages table.
- When an admin clicks **View**, the payload executes in their session.

Example request (redacted)

```
POST /graphql?_q=CreateInternalPreparedMessage HTTP/2
Host: REDACTED_TRIGGER_HOST
Referer: https://REDACTED_ADMIN_HOST/
Origin: https://REDACTED_ADMIN_HOST
Authorization: Bearer REDACTED_TOKEN
Content-Type: application/json
Platform-Code: REDACTED

{
  "operationName": "createInternalPreparedMessage",
  "query": "mutation createInternalPreparedMessage($input: CreateInternalPreparedMessageInput!)
{ createInternalPreparedMessage(input: $input) }",
  "variables": {
    "input": {
      "id": "REDACTED_MESSAGE_ID",
      "icon": "MESSAGE",
      "popup": true,
      "title": "test",
      "content": "<img src=x onerror=alert('DEMO_XSS')>",
      "recipients": {
        "filter": { "id": { "in": ["REDACTED_USER_ID"] } }
      }
    }
  }
}
```

Impact

- Session/token theft from browser storage can take over privileged backoffice accounts that view the message.
- Persistent XSS remains until the message is removed or sanitized.
- Pivot risk: act as the victim admin, change settings, read sensitive data, or escalate via UI actions.

Remediation (recommended)

- Encode all message fields on render; disallow raw HTML unless strictly sanitized.
- Treat internal messaging as an untrusted input channel between privileged users.
- Tighten WAF/API validation for script sinks; add CSP and HttpOnly session cookies where applicable.
- Audit “View” templates for unsafe HTML injection points.

End of redacted sample. Original evidence screenshots, Drive links, live hostnames, and exfiltration URLs intentionally omitted. Contact Primotonic for scoped assessments.

primotonic.com · hello@primotonic.com